

Sistemas Compostos de IA: Otimização de Respostas em Benchmarks Objetivos Através de uma Arquitetura Híbrida de Inferência e Validação com Multi-Agentes, Ferramentas e Orquestração Cognitiva

Hugo Emílio Nomura, Pedro Henrique Correia de Oliveira, Pedro Henrique Satoru Lima Takahashi, Vitor Monteiro Vianna

Ciência da Computação – Centro Universitário FEI, campus São Paulo

unifhnomura@fei.edu.br, unifpoliveira@fei.edu.br, unifptakahashi@fei.edu.br, unievvianna@fei.edu.br

Orientador: Charles Henrique Porto Ferreira

Departamento de Ciência da Computação, Centro Universitário FEI

cferreira@fei.edu.br

Resumo: Os *Large Language Models* (LLMs) têm, há algum tempo, deixado de ser apenas modelos voltados à geração de texto, consolidando-se como softwares e ferramentas completas. Essa evolução amplia significativamente seu escopo de aplicação, viabilizando funcionalidades como “pensamento prolongado”, acesso à internet e até mesmo suporte ao desenvolvimento de software por meio de interfaces especializadas, frequentemente referidas como *harnesses*, como CLIs — a exemplo de *Cloud Code* [1], *Codex* [2] e *OpenCLI* [3] — e *Integrated Development Environment* (IDEs), como *Cursor* e *Windsurf*. Em comum, essas soluções utilizam técnicas já consolidadas na literatura, muitas delas conhecidas há anos, como o princípio de “dividir para conquistar”, além de releituras dessas abordagens, com o objetivo de aproximar os resultados gerados do que é esperado, seja na construção de artefatos mais complexos ou na produção de respostas textuais mais precisas e alinhadas ao usuário. Nesse contexto, este estudo propõe a construção e análise desse conjunto de mecanismos e ferramentas que operam “ao redor” dos modelos de LLM, buscando verificar, na prática, se essas estratégias realmente melhoram a qualidade das respostas geradas. Para isso, serão utilizados como benchmark conjuntos de questões de múltipla escolha que abrangem diferentes áreas do conhecimento, como história, matemática, lógica, filosofia e relações públicas, desde níveis básicos de escolaridade até tópicos avançados em nível de doutorado (PhD).

Palavras-chaves: Esperar até o fim do desenvolvimento do glossário, e depois colocar aqui

I. Introdução

Os modelos de linguagem de grande porte (*Large Language Models* — LLMs) deixaram de ser meramente geradores de texto e hoje funcionam como plataformas compostas por camadas de processamento, coordenação e ferramentas auxiliares. Essa transição viabiliza capacidades como raciocínio em múltiplas etapas, orquestração de subagentes e uso de ferramentas externas, porém também revela limitações estruturais: nem sempre a melhoria observada no comportamento final advém apenas do modelo base, mas das arquiteturas que organizam e abstraem o processo de inferência.

Este trabalho parte de dois problemas centrais observados na prática e documentados na literatura: (1) a aparente discordância entre respostas iniciais e respostas revisadas por um mesmo modelo — uma sensação comum em interações com modelos mais antigos (por exemplo, no período de popularização do GPT-3.5) de que perguntas de confirmação como “Tem certeza?”, “revise sua resposta” ou mesmo o simples apontamento de erro (“você errou”) induziam o modelo a corrigir-se —, e (2) a forma como fluxos de pensamento (prompting encadeado, agentes e pipelines) são atualmente implementados, incluindo suas fragilidades frente ao crescimento do contexto.

O primeiro problema decorre do fato de que, em tare-

fas não-triviais, a resposta “de primeira” tende a ser apenas uma primeira tentativa, sujeita a erros e omissões, que pode ser significativamente aprimorada quando a inferência é estruturada em fases explícitas e curtas, com feedback intermediário (muitas vezes com quebra ou reinicialização parcial do contexto entre etapas). Trabalhos como *Chain-of-Thought* [4] e *Tree of Thoughts* [5] mostram ganhos ao induzir o modelo a produzir e avaliar passos intermediários; *ReAct* [6] evidencia a utilidade de alternar raciocínio e ações/observações; e linhas mais diretamente iterativas, como *Reflexion* [7] (reflexão após julgamento externo) e abordagens de *Self-Debugging* [8] (correção guiada por testes), demonstram que ciclos de tentativa–avaliação–revisão podem elevar a taxa de acerto ao transformar “repetir a pergunta” em um mecanismo sistemático de melhoria. Em paralelo, métodos de decomposição e planejamento — como *Least-to-Most Prompting* [9], *Decomposed Prompting* [10] e *Plan-and-Solve Prompting* [11] — reforçam empiricamente que quebrar objetivos complexos em subtarefas, separar planejamento de execução e limitar o escopo contextual de cada passo reduz erros de lógica e omissões de etapas, especialmente quando combinado com verificação automática. Assim, a melhoria não se explica apenas por “mais tokens” ou por uma chamada mais longa, mas pelo desenho de um pipeline que controla o que entra no

contexto, quando o modelo planeja, e como a resposta é validada e reescrita.

O segundo problema refere-se às práticas correntes de “thinking” em LLMs e às limitações impostas por janelas de contexto longas. Em arquiteturas populares de agentes, como *ReAct* [6] (*Synergizing Reasoning and Acting*) já mencionado, o modelo tende a operar acumulando em uma única janela um histórico crescente de raciocínios, ações e observações — um desenho que facilita o encadeamento narrativo do processo, mas que também amplia o risco de degradação conforme o volume de contexto aumenta. Esse tipo de limitação é consistente com evidências empíricas sobre o uso de contextos longos: em “*Lost in the Middle*” [12] (Liu et al.), por exemplo, observa-se degradação de desempenho em cenários nos quais o modelo precisa recuperar e utilizar informações relevantes em sequências extensas, especialmente quando esses trechos ficam “perdidos” no meio do contexto. Na prática, a comunidade de engenharia costuma descrever esse acúmulo progressivo e suas consequências como “esgotamento de contexto”, “compactação” e “context rot”: à medida que o histórico cresce, parte do conteúdo relevante perde saliência, compete por atenção com ruído e pode ser “diluída” por resumos implícitos, levando a omissões, inconsistências e aumento de alucinações.

Frente a essas propostas, adotamos como referência prática a implementação do *Ralph Wiggum Loop* popularizada por Geoffrey Huntley [13] no contexto de desenvolvimento de software, na qual um agente é conduzido por um ciclo rígido de execução e verificação para produzir artefatos corretos (p. ex., código) sob restrições operacionais claras. O objetivo deste trabalho é adaptar essa arquitetura — originalmente aplicada a tarefas de engenharia de software — para investigar se os mesmos princípios melhoram a resolução de questões objetivas em benchmarks de múltipla escolha. Em termos gerais, o *Ralph Wiggum Loop* pode ser entendido como um pipeline em que (i) ocorre uma quebra inicial do objetivo em subtarefas e fases, e (ii) cada fase é executada de maneira isolada e “stateless”, isto é, sem carregar diretamente o histórico completo da fase anterior: em vez de continuidade conversacional, o sistema preserva apenas artefatos controlados (como instruções, respostas intermediárias ou resultados de validação) e reinicializa o contexto a cada iteração (“*Fresh Context*”). Embora o nome e o formato operacional sejam recentes, a lógica do loop se apoia em linhas acadêmicas já discutidas — decomposição (*Least-to-Most* [9]; *Decomposed Prompting* [10]), separação entre planejamento e execução (*Plan-and-Solve* [11]) e ciclos de reflexão/autodepuração guiados por avaliação externa (*Reflexion* [7]; *Self-Debugging* [8]) — oferecendo um arcabouço pragmático para aplicar esses achados em orquestrações reproduzíveis.

Neste trabalho, adotamos a hipótese de que a qualidade objetiva das saídas em tarefas complexas depende tanto do LLM base quanto do “*Harness*” que governa seu uso. Para testar essa premissa, nossa meta é avaliar o impacto da orquestração estruturada — inspirada no *Ralph Wiggum Loop* [13] — frente à inferência direta tradicional. A avali-

ação em andamento baseia-se em benchmarks acadêmicos rigorosos, visando demonstrar se uma arquitetura composta por etapas de decomposição e verificação iterativa é capaz de elevar o desempenho de modelos com menos parâmetros a patamares competitivos, superando abordagens lineares em métricas de acurácia.

II. Representação do processo

A comparação entre o uso direto de um modelo de linguagem e o uso do mesmo modelo dentro de uma arquitetura orquestrada constitui o núcleo conceitual deste trabalho. Por essa razão, a representação do processo não deve funcionar apenas como elemento ilustrativo, mas como uma forma de explicitar, desde o início, a hipótese central do estudo: a qualidade da resposta produzida por um LLM pode variar não apenas em função do modelo utilizado, mas também em função da organização da inferência, das etapas intermediárias adotadas e dos mecanismos de controle inseridos entre a entrada e a resposta final [4], [5], [14].

Na configuração direta, a tarefa é enviada ao modelo como uma única instrução, e a resposta é produzida em uma única geração. Essa forma de uso tende a concentrar, em um único passo, operações distintas como interpretação do enunciado, definição da estratégia de resolução, checagem de restrições e escolha da alternativa [15], [16]. Embora essa abordagem seja eficiente para tarefas simples, ela oferece pouco controle sobre o processo intermediário de raciocínio [4]. Já na configuração orquestrada, essas operações são explicitadas e distribuídas em etapas independentes, permitindo maior visibilidade e controle sobre como a resposta é construída [7], [17].

A Figura 1 sintetiza essa distinção em nível macro. Em vez de tratar a resposta como consequência imediata de uma única chamada ao modelo, a arquitetura proposta a representa como resultado de uma sequência organizada de análise, decomposição, processamento, integração e validação [13], [18]. Esse contraste é importante porque estabelece, visualmente, a diferença entre o *baseline* do experimento e a condição principal que será avaliada ao longo do trabalho.

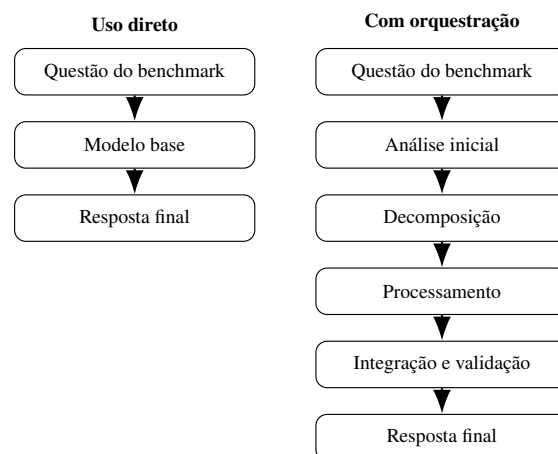


Figura 1. Comparação entre o fluxo direto de inferência e o fluxo orquestrado adotado como arquitetura experimental.

Embora a comparação geral seja suficiente para diferenciar as duas condições experimentais, ela ainda não mostra como a arquitetura proposta reorganiza internamente a resolução de uma tarefa. A literatura sobre agentes de linguagem indica que ciclos iterativos de planejamento, execução e reflexão são superiores à geração monolítica em tarefas de múltiplas etapas [5], [7]. Por isso, a Figura 2 detalha a lógica de transformação da questão em resposta, destacando que o processo intermediário não é um acréscimo meramente estético, mas uma estratégia de estruturação da inferência [14], [17]. Nessa perspectiva, a tarefa deixa de ser tratada como um bloco indivisível e passa a ser abordada como uma sequência coordenada de operações menores, cada uma com função específica dentro do pipeline [18].

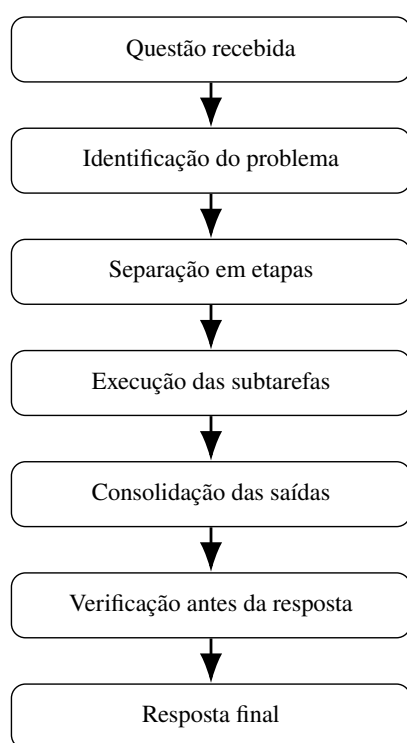


Figura 2. Representação sequencial das etapas que estruturam a passagem da questão à resposta final.

Essa representação torna a seção mais útil ao artigo porque ela deixa de ser apenas uma abertura visual e passa a introduzir, de maneira argumentativa, o objeto metodológico que será aprofundado a seguir. A decomposição em etapas menores está alinhada a resultados experimentais que mostram ganhos de desempenho quando o raciocínio é explicitado antes da escolha da alternativa final [4], [15], [17]. Assim, a representação do processo contribui para reduzir ambiguidades sobre o que exatamente será comparado no experimento e prepara a leitura da metodologia de forma mais coerente.

III. Metodologia

A metodologia deste trabalho foi construída para isolar o efeito da arquitetura de inferência sobre a qualidade das respostas produzidas por um modelo de linguagem. Em vez de comparar modelos distintos entre si, a estratégia central

consiste em manter o modelo base constante e alterar a forma como a tarefa é organizada ao longo da execução [19], [20]. Esse desenho experimental é relevante porque permite analisar se diferenças de desempenho decorrem da capacidade intrínseca do modelo ou da estrutura adotada para conduzir o raciocínio e a produção da resposta final [21], [22].

De modo geral, o experimento compara duas condições principais. A primeira corresponde ao uso direto do modelo, no qual a questão é submetida ao LLM em uma única chamada [15]. A segunda corresponde ao uso do mesmo modelo dentro de um pipeline estruturado, inspirado no *Ralph Loop* [13], em que a resolução da tarefa é distribuída em etapas menores e complementada por verificação intermediária e integração de ferramentas auxiliares [23], [24]. Essa organização foi adotada porque dialoga com a literatura sobre autoaperfeiçoamento iterativo e decomposição de tarefas complexas, que demonstra ganhos quando a inferência incorpora planejamento, revisão e controle parcial [7], [14], [17].

A. Desenho experimental

O desenho experimental parte de uma lógica comparativa rigorosa: a mesma questão será submetida às diferentes condições de execução, preservando-se o mesmo modelo subjacente [25], [26]. Isso permite que a variável principal do estudo seja a arquitetura de processamento, e não a troca do modelo [19]. Em outras palavras, o trabalho busca observar o quanto a organização da inferência interfere no resultado obtido quando o núcleo gerador permanece constante [20], [22].

A Figura 3 resume esse princípio. A partir de uma mesma entrada, duas rotas são construídas: uma rota de referência, mais simples, e uma rota principal, mais estruturada. Ao final, ambas produzem uma resposta comparável com o gabarito da questão, o que permite levar a comparação adiante na etapa de avaliação dos resultados [25], [26].

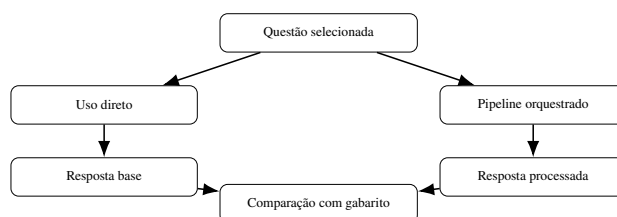


Figura 3. Desenho experimental comparando uso direto e pipeline orquestrado com a mesma base de questões.

Essa forma de organização favorece a interpretação dos resultados porque reduz um problema comum em comparações com LLMs: atribuir ganhos ou perdas ao modelo quando, na realidade, parte do efeito pode estar na maneira como a tarefa foi estruturada [19], [21]. Ao delimitar explicitamente a arquitetura como variável de interesse, o estudo procura oferecer uma análise mais precisa do papel da orquestração na qualidade da resposta [22].

B. Arquitetura proposta

A arquitetura central deste trabalho baseia-se no *Ralph Loop* [13], adotado aqui como uma estratégia de execução em ciclos e etapas mais isoladas, o que reduz a dependência do acúmulo de contexto ao longo do processo. O *Ralph Loop* funciona como um padrão de orquestração cíclico no qual o modelo de linguagem atua de maneira análoga a um agente autônomo, avançando na tarefa por meio de uma interação contínua entre leitura, planejamento, execução e verificação do contexto atual. Em vez de deduzir a resposta em uma via de mão única, o loop reavalia iterativamente as soluções parciais produzidas, formula os próximos passos necessários e testa sua própria lógica até acumular confiança suficiente de que a tarefa original foi resolvida e a execução pode ser finalizada (*Halt*).

Conceitualmente, essa escolha aproxima o experimento de métodos de raciocínio estruturado e de orquestração deliberada, nos quais resolver um problema exige organizar o fluxo de processamento passo a passo [5], [7], em vez de apenas gerar uma resposta direta após o prompt inicial [15].

Na prática, o pipeline proposto recebe a questão e faz uma análise prévia para identificar o tipo de problema, a área abordada e as restrições da tarefa [18]. Como o enunciado já foi interpretado nessa fase inicial, a resolução passa a ser dividida em ações menores e mais focadas, como: estruturar um plano de raciocínio, levantar as informações necessárias, checar a consistência lógica e validar a alternativa correta. Essa divisão está alinhada à ideia de auto-refinamento iterativo, em que resultados parciais são revisados antes de compor a saída final [14], [17]. As respostas geradas nessas etapas parciais são unidas, podendo ainda passar por um filtro extra de verificação antes de serem registradas [27], [28].

A Figura 4 ilustra essa dinâmica. O objetivo do diagrama não é impor uma regra rígida de execução, mas expor a lógica geral da arquitetura: uma série organizada de ações menores, desenhadas para diminuir a incerteza da inferência e garantir mais controle sobre a resposta gerada [7], [13].

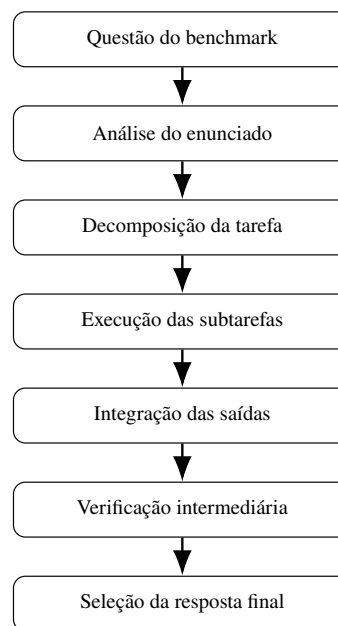


Figura 4. Fluxo da arquitetura proposta, desde a entrada da questão até a consolidação da resposta final.

Essa decomposição tem relevância metodológica porque evita concentrar, em uma única geração, decisões heterogêneas que exigem competências distintas [4], [5]. Em uma questão de física, por exemplo, a identificação das grandezas relevantes não é a mesma operação da escolha final da alternativa correta; em uma questão de filosofia, reconhecer a tese central também não equivale à comparação entre alternativas. Ao explicitar essas diferenças em etapas menores, o pipeline procura alinhar a execução do modelo a uma lógica de resolução mais estruturada [14], [22].

C. Componentes auxiliares

Embora a arquitetura principal já seja suficiente para estabelecer a comparação central do artigo, o pipeline poderá incorporar componentes auxiliares voltados ao fortalecimento da resposta final. Entre esses componentes, destacam-se mecanismos de verificação intermediária, nos quais uma saída parcial pode ser inspecionada antes de ser aceita como resposta consolidada [27], [28], e ferramentas externas de apoio integradas via protocolos estruturados [23], [24]. Esses elementos não constituem o objeto principal da hipótese experimental, mas ampliam a capacidade de controle e fundamentam-se em evidências recentes de que agentes com acesso a ferramentas superam modelos isolados em tarefas que exigem consultas externas [18], [29].

A Figura 5 apresenta uma visão mais estrutural do pipeline, destacando a relação entre o núcleo de execução, o módulo de verificação e os recursos auxiliares. Essa representação evidencia que a arquitetura proposta não é apenas uma cadeia linear de passos, mas um arranjo em que diferentes componentes apoiam a produção e a validação da resposta [7], [13].

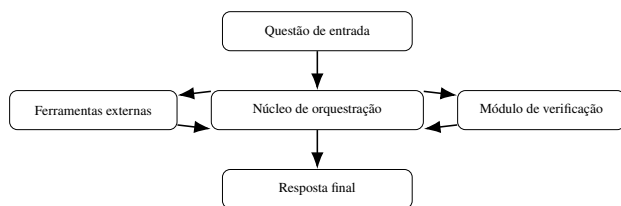


Figura 5. Visão estrutural da arquitetura proposta, com o núcleo de orquestração interagindo com módulos de ferramentas e verificação.

A inserção desses módulos auxiliares ajuda a explicar por que a arquitetura proposta deve ser compreendida como uma composição de etapas e não apenas como um prompt maior ou mais detalhado. O ganho esperado, caso ele exista, não decorre exclusivamente da redação da instrução inicial, mas da possibilidade de distribuir funções diferentes entre partes distintas do sistema, permitindo revisão, controle e apoio externo durante a execução [14], [23], [24].

IV. Plano de avaliação

O plano de avaliação visa mensurar o impacto da arquitetura de orquestração na acurácia de modelos de linguagem em tarefas de raciocínio geral, comparando sistematicamente a inferência direta com a orquestrada. Para essa validação, a metodologia contempla cinco modelos selecionados para cobrir uma progressão de complexidade: Llama 3.1 (8B, 70B e 405B), Claude Sonnet 4.6 e GPT 5.4.

Para cada modelo, está prevista a comparação de duas configurações: (i) uma inferência "linear", realizada por meio de uma requisição direta ao modelo sem processamento auxiliar, e (ii) a mesma inferência mediada pelo *Harness* proposto, empregando ciclos de verificação e ferramentas externas. O desempenho será medido por meio de 180 questões amostradas das bases GPQA [26], MMLU [25] e MMLU-Pro [30], escolhidas por seu elevado rigor acadêmico. A métrica primária consistirá na acurácia, obtida pela correspondência estrita entre a alternativa final selecionada pelo sistema e o gabarito.

A execução seguirá um protocolo no qual, inicialmente, os modelos estabelecem seu desempenho de base na inferência padrão. Na sequência, as métricas serão reavaliadas sob a arquitetura orquestrada (inicialmente focada na família Llama), sempre respeitando restrições operacionais *stateless*. O objetivo esperado desta avaliação é verificar se o *harness* altera o posicionamento dos modelos no ranking global, testando a hipótese de que a orquestração estruturada suplanta ganhos atrelados puramente à escala de parâmetros.

A Tabela I apresenta as projeções de acurácia que fundamentam este plano e o benchmark pretendido para a validação da arquitetura.

Tabela I. Projeção de acurácia média esperada por modelo e configuração (dados hipotéticos para fins de planejamento).

Modelo	Configuração	Acurácia Esperada
Llama 3.1 8B	Direto	52%
Llama 3.1 70B	Direto	74%
Llama 3.1 8B	Ralph Loop	77%
Llama 3.1 405B	Direto	84%
Llama 3.1 70B	Ralph Loop	88%
Claude Sonnet 4.6	Direto	91%
Llama 3.1 405B	Ralph Loop	94%
GPT 5.4	Direto	97%

V. Referências

- [1] Anthropic, *Cloud Code*, <https://www.anthropic.com/>, 2026.
- [2] M. Chen et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [3] *OpenCLI*, <https://opencli.ai/>, 2026.
- [4] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *arXiv preprint arXiv:2201.11903*, 2022.
- [5] S. Yao et al., "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," *arXiv preprint arXiv:2305.10601*, 2023.
- [6] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2022.
- [7] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan e S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," *arXiv preprint arXiv:2303.11366*, 2023.
- [8] X. Chen, M. Lin, N. Schärli e D. Zhou, "Teaching Large Language Models to Self-Debug," *arXiv preprint arXiv:2304.05128*, 2023.
- [9] D. Zhou et al., "Least-to-most prompting enables complex reasoning in large language models," *arXiv preprint arXiv:2205.10625*, 2022.
- [10] T. Khot et al., "Decomposed prompting: A modular approach for solving complex tasks," *arXiv preprint arXiv:2210.02406*, 2022.
- [11] L. Wang et al., "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," *arXiv preprint arXiv:2305.04091*, 2023.
- [12] N. F. Liu et al., "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, v. 12, pp. 277–294, 2024.
- [13] G. Ghuntley, *Ralph Loop: A Structured Orchestration Pattern for LLM Inference*, Online. Disponível em: <https://ghuntley.com/specs/ralph/>, Acesso em: abr. 2026, 2025.

- [14] A. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” *arXiv preprint arXiv:2303.17651*, 2023.
- [15] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo e Y. Iwasawa, “Large Language Models are Zero-Shot Reasoners,” *arXiv preprint arXiv:2205.11916*, 2022.
- [16] T. B. Brown et al., “Language Models are Few-Shot Learners,” em *Advances in Neural Information Processing Systems*, 2020.
- [17] X. Wang et al., “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” *arXiv preprint arXiv:2203.11171*, 2023.
- [18] Y. Shen, K. Song, X. Tan, D. Li, W. Lu e Y. Zhuang, “HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face,” em *Advances in Neural Information Processing Systems*, 2023.
- [19] L. Zhang et al., “Enhancing Large Language Model Performance To Answer Questions and Extract Information More Accurately,” *arXiv preprint arXiv:2402.01722*, 2024.
- [20] Y. Liu et al., “Beyond Prompt Content: Enhancing LLM Performance via Content-Format Integrated Prompt Optimization,” *arXiv preprint arXiv:2502.04295*, 2025.
- [21] OpenAI, “GPT-4 Technical Report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [22] A. Huttula, “Advanced Prompt Engineering: Systematic Approaches to Enhance LLM Performance,” diss. de maestr., Jamk University of Applied Sciences, 2025.
- [23] Anthropic, “Model Context Protocol (MCP): Enabling Tool Use and External Memory in Language Model Pipelines,” *Technical Report*, 2025, Disponível em: <https://modelcontextprotocol.io>.
- [24] A. Sarkar, P. Gupta e R. Mehta, “Integrating External Tools with Large Language Models via Structured Protocols,” *arXiv preprint arXiv:2503.11200*, 2025.
- [25] D. Hendrycks et al., “Measuring Massive Multitask Language Understanding,” em *International Conference on Learning Representations*, 2021.
- [26] D. Rein et al., “GPQA: A Graduate-Level Google-Proof Q&A Benchmark,” *arXiv preprint arXiv:2311.12022*, 2023.
- [27] L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” em *Advances in Neural Information Processing Systems*, 2024.
- [28] D. Li et al., “From Generation to Judgment: Opportunities and Challenges of LLM-as-a-Judge,” *arXiv preprint arXiv:2411.16594*, 2024.
- [29] Y. Chen et al., “TUMIX: Multi-Agent Test-Time Scaling with Tool-Use Mixture,” *arXiv preprint arXiv:2510.01279*, 2025.
- [30] Y. Wang et al., “MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark,” *arXiv preprint arXiv:2406.01574*, 2024.